Child Mind - Report

Filippo Pizzo 894982

1. Introduction

The goal of this project is to develop a predictive model capable of analyzing children's physical activity data to detect early indications of problematic internet and technology use.
To achieve this goal we have 2 datasets: *train.csv* and *test.csv*, containing tabular data about measurements from various instruments.
In this report I will first describe the data cleaning process and then present the 2 models I used to accomplish the classification task, in the meantime I will explain my decision and the considerations I've made based on the obtained results.

2. Data Processing

In this section, I will explain what I did to obtain a dataset appropriate for training the models.
In the beginning, I took a look at the data I had to work on, and thanks to that I collected useful insights into the dataset:

- The train.csv file contains more features than the test.csv file.
- The feature that has to be predicted has 4 possible values, so what we have to compute is a multi-class classification. I've also noticed that in some of the instances the label is missing.
- Most of the features have *NaN* as one of their possible values, which means that we have to deal with missing values.
- In opposition to the last point, the Demographics information, that are age and sex, are always present in the dataset.
- We have 3 types of data in the dataset: *int*, *float*, *string*, so we have to deal with both numerical and categorical variables.
- All the *string* type data represent the seasons, so they have only 4 possible values.
- There is an identifier feature which is unique.

I will use all this information during the cleaning process to obtain a robust dataset.

2.1 Dealing with Not Useful Data

I proceed by dealing with the not useful columns and rows, which I will remove to get a more consistent and less noisy dataset.
The features that are present only in the training dataset are the one regarding the Parent-Child Internet Addiction Test, which measures characteristics and behaviors associated with compulsive use of the Internet. From the description of this group of features, we can understand that they could be very useful for the prediction, but we will delete them since they don't provide any help for the classification.
I will then remove the rows where the value of the label is *NaN* since this is the field required for supervised learning.
Next, I will remove the identifier feature since as we already said, it is unique so not relevant for a classification task.
Subsequently, I will drop the instances that present anomalies in their values, for example where weight is equal to 0 which is impossible.
We can now start dealing with the missing values.

2.2 Dealing with Missing Values

I've noticed that in the dataset we have a group of Physical Measures like height, BMI and weight, that for a child are related mostly to age and sex. Since this pair of features is never *NaN* in our dataset, I thought that we could use them to insert in the rows where the physical measures are missing, the average values for the specific age and sex of the child.
This computation is only made in the rows where all the physical measures are missing, since in case some of them are present, using them to predict the other through imputation is a better idea.
To accomplish this I used an external source: a csv file that I've created using the NHANES data that assess the health and nutritional status of children in the United States.
I decided to use this approach for the physical measures because it limits the distortion of the measures since a 7 year old girl is not going to be as tall and as heavy as a 16 year old boy.
For the rest of the data I decided to:

- drop the instances above a certain percentage of *NaN* values

- replace the numerical missing values with the KNN Imputer
- replace the categorical missing values with the mode

I dropped instances with over a certain threshold of missing values to keep the integrity and the coherence of the model, since the quantity of information they provide is too poor to be useful for training the models. I selected as a threshold 65% of *NaN* values, which is a high value, but since we have a lot of missing values in our dataset, we don't want to lose too many rows, so I prefer to remove only the extremely poor instances.
Instead, I haven't dropped the features above a certain threshold of missing values because we don't know beforehand the importance of the features and they could still be useful.
Before performing the other 2 parts of this subtask, I split the dataset into train and test sets, in this way, I prevent potential bias in the imputation of the missing values.
To deal with the numerical features I replaced the *NaN* values of an instance with the values of the closer neighbors using the *KNN Imputer*. This technique is better than using the mean because it takes into account the similarity between instances, allowing imputed values to reflect the underlying structure of the data, and this is important for a heterogeneous dataset where children can have very different characteristics.
For what concerns categorical data we have to deal with 2 computations: replace missing values with the mode and then transform them with One-Hot Encoding.
In this case, I opted for the mode and not a more complex imputation because season is a low-cardinality variable and relatively evenly distributed, imputation with the mode provides a simple yet effective approach.

2.3 Dealing with Outliers

Now that we have handled all the missing values, we want to know if there are some outliers in our dataset that could damage the data integrity.
I have detected the outliers using the Random Forest as Similarity Estimator technique, indeed we can compute the similarity of two instances by checking the fraction of leaves reached by both the instances traversing the forest. Then we can use this measure to detect which are those instances that are dissimilar from the other points in the dataset, by computing the outlier score.
The results of this process reveal that even the instances with the higher outlier score are quite normal upon inspection, so we can't consider them as anomalies. I have also noticed that the instances with the higher outlier score are mostly male adolescents and since they are not underrepresented in the dataset, this could mean that it depends on the way the Random Forest has used data for training. So in this last part of the data processing, I didn't modify the dataset.
In conclusion, we have a complete dataset, without anomalies and with encoded categorical data, that is now ready to be used for a classification task.

3. Random Forest

I've chosen Random Forest to accomplish this task due to its robustness and performance advantages:

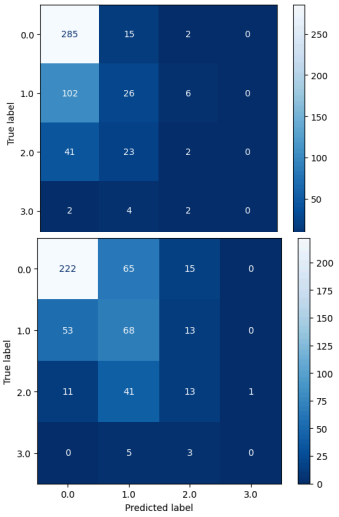
- It exploits bagging to aggregate multiple fully-grown decision trees, which reduces variance and helps preventing overfitting
- Each tree is built on a small subset of features increasing the diversity among tree, this improves the generalization

At first, I computed the baseline accuracy, which represents the accuracy of a naive classifier that assigned every instance to the most frequent label of the train set. Our goal is to construct a model that predicts at least better than the naive classifier, and since this is a multi-label classification task we also want a model able to predict every label and a quite good generalization.
The first model I have constructed is a basic Random Forest with no parameter specified, this model is the one I used as a benchmark against which I can compare an optimal model.

3.1 Tuning the Hyper-Parameter

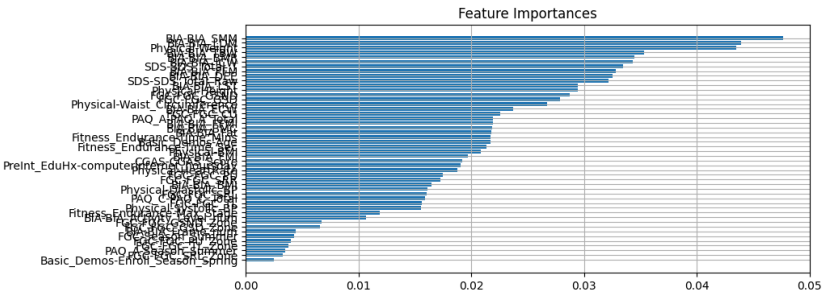
To obtain the optimal model, I fine-tuned its hyper-parameters using a grid search.
The grid search allowed me to perform the validation by selecting some possible values for the parameters and training a model for each combination of them to then compare the models and automatically select the one with the best validation accuracy score.

The grid search is a k-fold cross-validation, so this means that the hyperparameter choice is less affected by some specific values since for each model we compute the training 5 times each time with a different subset as the validation set.
On this basis, I thought that it was the best technique to compute the tuning process.
Now that I got a tuned model I decided to look at the confusion matrix and investigate the performance more specifically.
We can see that the model predictions are quite imbalanced since more than 80% of the instances are predicted as class 0.
This is normal since class 0 is the one with more instances, but still, this percentage is way higher than the baseline accuracy (the percentage of class 0 instances in the test set).
Since our goal is to get a model capable of predicting instances of each label, we want to mitigate the bias toward the dominant class, to do so I gave the classes an inversely proportional weight to their frequency.
As we can see with this approach, the model is able to also predict instances that are not from class 0.
Another thing we can notice is that instances of classes 2 and 3 are still difficult to predict since they are few, but this time their predictions are allocated in a more similar class.
Since it's more generalized and still better than the naive classifier, we will use this model instead of the only tuned one.



3.2 Feature Subset Selection

For now, we have a model that uses all the features present in the dataset and since in the dataset we have many features, some of them could be not really useful and only slow down the model.
In this section, our goal is to get a more generalized model and reduce overfitting. To accomplish this I selected a subset of the most informative features to train the model on.
For the selection of the features, I performed a Recursive Elimination in a cross-validation loop.
It's a quite slow method, but more precise and less affected by dependencies and correlation than the embedded approach since it removes few features at time and revalidates the model performance each time.
After computation, the number of features dropped from 88 to 43, this suggests that a good part of the original features were not informative or redundant, so dropping them allowed our model to focus on the more useful features for the prediction.
The confusion matrix is quite similar to the one of the tuned balanced model so we didn't lose prediction power by removing half of the features.
If we take a look at the importance of the features we kept, we can see that the most important features are all numerical, as we could imagine the categorical features weren't useful for the prediction since they were just informing us on the seasons the measurements were made.



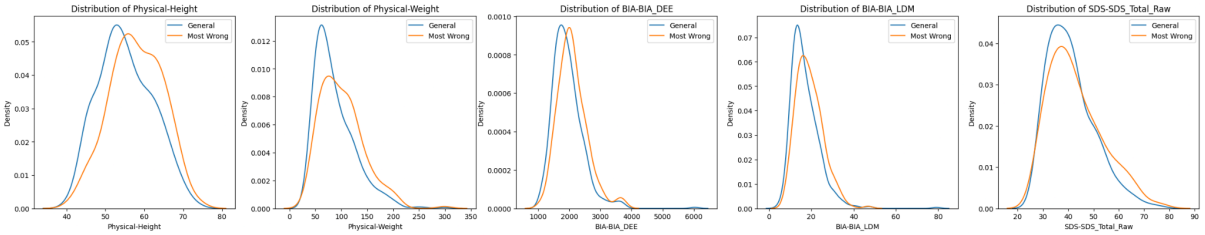
We can also see that the most important features are relative to 3 groups:

- Bio-electric Impedance Analysis: measure of key body composition elements, like BMI and fat
- Physical Measures
- Sleep Disturbance Scale: scale to categorize sleep disorders in children

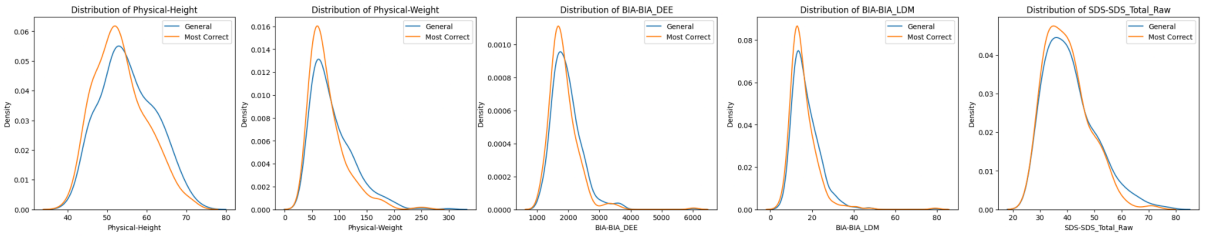
After the construction of the model, I evaluated it through the *test.csv* for the Kaggle competition.
I processed the data like I did with the *train.csv* and used the fine-tuned parameters model with the subset of features to predict the labels. As a result, I got the maximum private score of 0,411.

3.3 Investigate Problematic Instances

In this task, I analyzed the instances with the most wrong predictions and the most correct predictions.
For most wrong predictions I mean the instances that the model was very confident were of a label, but are of another one, instead for most correct predictions I mean the instances that were easier to predict by the model.
The most wrong predictions are mostly about instances where the true label was 1 but the model predicted 0.
Instead, the most correct instances are about instances where the label is 0.
This is probably due to the fact that class 0 is the one with more instances and even if we have assigned weight to the classes to reduce the unbalance, we still have to deal with it.
I compared the 500 most wrong instances to all the instances, aiming to see possible differences in the behavior. Since I couldn't compare the instances on every feature, I decided to select the most important features because they are the ones that have more impact on the model.



For what concerns *Physical_Height*, the distribution of the wrong instances is shifted to the right of the general distribution, this suggests that higher children are more likely to be subject to misclassification and so to have a problematic use of internet since the most misclassificated instances are the one with a label of 1 or higher.
Furthermore, in *Physical_Weight* we have quite the same difference in the distribution with a higher density of wrong prediction on the left of the peak of the normal distribution, but this time the wrong distribution is also wider. The two physical features have a similar distribution because they are correlated since a higher child usually weighs more.
For the BIA features we got that the most wrong distribution is slightly shifted to the left, meaning that higher measures of key body composition elements are more difficult to predict.
Finally, for SDS_Total_Raw the incorrectly predicted instances have a wider distribution.
These findings could suggest a possible correlation between the two Physical and BIA measures since the most wrong instances are shifted to the right for all 4 distributions.
In the analysis of the most correct predictions, we can see that their distributions are the opposite of the most wrong predictions distributions.



For what concerns physical height we can see that shorter children are easier to predict, this could be due to the fact that a shorter child is also younger and for a small kid it's difficult to have a high sii so it's easier to be correctly predicted by our model. The same reasoning applies to physical weight.

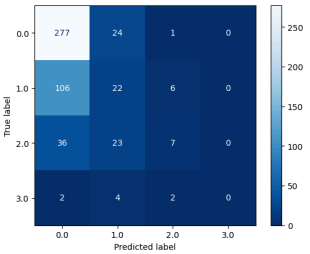
4. Cat Boost

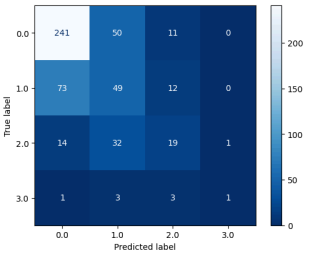
Cat Boost is an algorithm for gradient boosting on decision trees.
It is particularly interesting because it is designed to handle categorical data very efficiently and since they are present in our dataset, I thought it would be a good model to use.
Thanks to the fact that it's a boosting algorithm it can be pretty useful to reduce bias.
Since it automatically handles categorical data, this time we don't need to encode them, but just pass them as parameters of the model.

For the initial model, I trained a basic Cat Boost model just specifying as parameters the categorical features and a standard number of iterations.
A difference I noticed from the Random Forest model is that the predictions are more balanced by default, so the model is more capable of predicting all the classes without excessively focusing on label 0.

4.1 Tuning the Hyper-Parameter

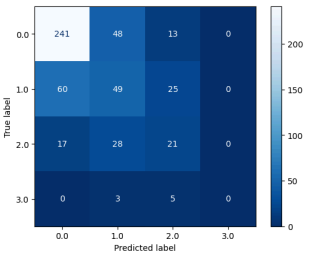
To obtain the optimal model, I fine-tuned its hyper-parameters automatically using a k-fold validation with grid search. I passed to the grid search the learning rate, the depth, the number of iterations and the L2 regularization that helps control model complexity.
I didn't fine-tune the loss function since we have 4 possible class labels and the correct one for this type of classification task is *MultiClass*. I also specified the early stopping rounds parameters that stop the iteration process of a model if it doesn't improve.
The fine-tuned model is more accurate than the basic one, but from the confusion matrix we can see that the model is less balanced, so I assigned weights to the labels. This time I didn't assign weights inversely proportional to the class frequency since the model is more balanced than the Random Forest, and assigning too much power to classes with very few instances could affect the accuracy of the model. So I decided to assign weights respectively of 1, 2, 4, 8 since after some tests, they were the best compromise between accuracy and balance of the model.
In this way, we get a more balanced model still with some problems in predicting instances with true label 0, but this is quite normal since there are very few instances so the model isn't properly trained on them.



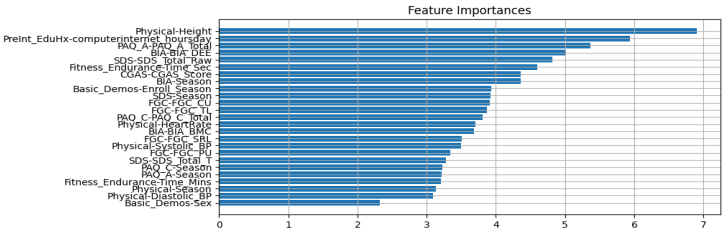


4.2 Feature Subset Selection

I selected a subset of the most informative features with the goal of adding robustness and reducing overfitting.
Since *RFECV* doesn't know how to handle categorical variables, this time I performed the selection using the embedded approaches with cross-validation. Starting with all features, I computed their importance and removed the least important features at every cycle until removing features would compromise the performance.
After selecting the features that give the best prediction, I trained the tuned model with weights on them and got a model with a very similar prediction ability, but that now benefits from a reduced complexity and reduced risk of overfit.



Taking a look at the most informative features, we can see that they are quite different from the one of the Random Forest model.
The categorical features have an importance, this is probably due the fact that in the random forest, they were splitted in 4 binary features, affecting their significance.



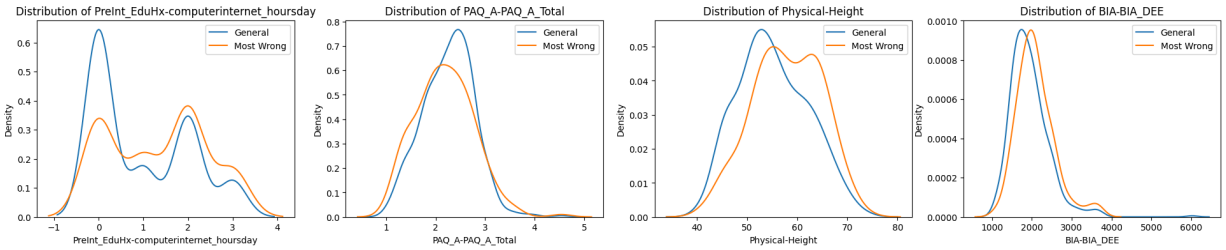
The most informative groups of measure changed, indeed Physical and SDS measures are still important, but this time we also got features relative to:

- Internet use
- Physical Activity Questionnaire: children participation in vigorous activities
- FitnessGram: measurements of cardiovascular fitness assessed using the NHANES treadmill protocol.

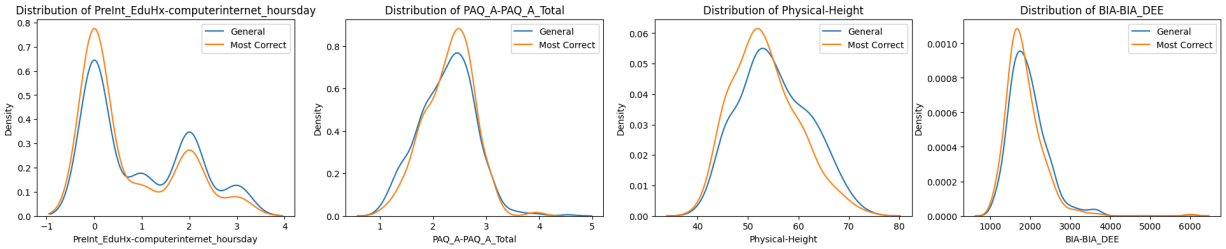
After the construction of the model, I evaluated it through the *test.csv* for the Kaggle competition.
I processed the data like I did with the *train.csv* and used the fine-tuned parameters model with the subset of features to predict the labels. As a result, I got the maximum private score of 0,432.

4.3 Investigating Problematic Instances

Even this time , the most wrong predictions are about instances where the true label was 1 but the model predicted 0.
Instead, the most correct instances are about instances where the label is 0.
This is probably due to the fact that class 0 is the one with more instances and even if we have assigned weight to the classes to reduce the unbalance, we still have to deal with it.
I focused on analyzing the most influential features of the classifier.



As for the RF model we have Height and BIA-DEE, but this time we also have the Activity Summary score of the adolescents and the daily hours of internet use.
As regards the most wrong predictions, the distribution of the daily hours of internet use shows a lower density than the general on the value 0 and a higher density on the others, suggesting that more hours of internet usage also means more probability of problematic use, so more difficulties to predict instances for our model.
Taking a look at the distribution of the activity summary score, we can see that the distribution of the most wrong predictions is wider than the general one with the peak shifted to the left, suggesting that children with a lower score and so a lower physical ability are more likely to have a more problematic use of internet since the most wrong predictions represents instances that at least have a slight problematic use of internet.
For what concerns Physical Height and BIA-DEE, the distribution is similar to the one of the RF model.
In the analysis of the most correct predictions, we can see that their distributions are the opposite of the most wrong predictions distributions.



5. Conclusions

This report explained the developing process of two machine learning models for a multi-class classification problem.
I encountered a few challenges due to the dataset like missing values and an unbalanced distribution of the label, which I faced by data imputation and different weights on the labels.
The Random Forest model proved a robust classificator capable of good performance and generalization after a careful hyperparameter tuning and feature selection. The use of weights significantly improved its ability to classify minority classes, and the analysis of feature importances highlighted the key role of physical and sleep-related features. Meanwhile from the most wrong predictions analysis we saw a bias related to the physical measures.
The Cat Boost model, with its native handling of categorical features and gradient boosting mechanism, provided a more balanced prediction. Additionally, the CatBoost model identified a slightly different set of important features, emphasizing not just physical characteristics, but also internet usage patterns and physical activity scores.
Overall, both models provide promising baselines for detecting problematic internet usage. The CatBoost model, in particular, shows potential for deployment in real-world settings due to its interpretability and balanced performance.